

COMPUTER ARCHITECTURE AND ORGANIZATION

COMPUTER MEMORY SYSTEM

Learning Objectives

At the end of this module, you should be able to:

- (1) Understand the main characteristics of computer memory systems and the use of memory hierarchy
- (2) Understand cache memory, and the key elements of cache design

Outline:

The Memory System:

- Memory Characteristics
- Semiconductor main memory
 - *Organization, DRAM and SRAM*
- ROM
 - *Types of ROM, Chip Logic, Chip packaging*
- Interleaved Memory
- The Memory Hierarchy
- Cache Memory Principles
 - *Elements of Cache Design*

Computer Memory

In computing, memory is a device or system that is used to store information for immediate use in a computer or related computer hardware and digital electronic devices. The term memory, in everyday usage is often synonymous with the term primary storage or main memory.

Computer Memory is one of the core components of the Computer Central Processing Unit (CPU). The concept of computer memory seems simple as there are ranges of types and each of the types can be categorized according to technology, organization, performance and cost. **The Computer memory is one of the determinant factors of the cost of a PC.**

The diverse nature of digital devices in terms of needs, make-ups and complexities does not allow for single technology memory types. Consequently, the typical computer system is equipped with a hierarchy of memory subsystems; some are internal to the system and some external.

The **internal** memories are directly accessible while the **external** memories are accessed via I/O modules.

Table 1: Memory Characteristics

Location	Performance
Internal (e.g., processor registers, cache, main memory)	Access time
External (e.g., optical disks, magnetic disks, tapes)	Cycle time
Capacity	Transfer rate
Number of words	Physical Type
Number of bytes	Semiconductor
Unit of Transfer	Magnetic
Word	Optical
Block	Magneto-optical
Access Method	Physical Characteristics
Sequential	Volatile/nonvolatile
Direct	Erasable/nonerasable
Random	Organization
Associative	Memory modules

The various characteristics of memory are as presented in Table 1 and describe as follows in the next section.

3.1 Memory Characteristics

1. **Location:** Refers to whether memory is internal or external to the computer. Internal memory is often equated with main memory, but there are other forms of internal memory; Processor requires its own local memory in the form of registers, control unit portion of the processor may also require its own internal memory. Cache is another form of internal memory. External memory consists of peripheral storage devices, such as disk and tape, which are accessible to the processor via I/O controllers.

2. **Capacity:** is the amount of data that can be stored in the memory storage unit. This storage capacity is expressed in terms of Bytes. For internal memory, this is typically expressed in terms of bytes (1 byte = 8 bits) or words. Common word lengths are 8, 16, and 32 bits. External memory capacity is typically expressed in terms of bytes or higher units of store.
3. **Unit of transfer:** For internal memory, the unit of transfer is equal to the number of electrical lines into and out of the memory module. This may be equal to the word length, but is often larger, such as 64, 128, or 256 bits.

The internal memory transfer rate is governed by the data bus width. For external memory, usually a block is much larger than a word Addressable. To clarify the units of transfer concept, we hereby define the following related terms:

- i. **Word:** The “natural” unit of organization of memory. The size of a word is typically equal to the number of bits used to represent an integer and to the instruction length. Unfortunately, there are many exceptions. For example, the CRAY C90 (an older model CRAY supercomputer) has a 64-bit word length but uses a 46-bit integer representation. The Intel x86 architecture has a wide variety of instruction lengths, expressed as multiples of bytes, and a word size of 32 bits.

- ii. **Addressable units:** In some systems, the addressable unit is the word. However, many systems allow addressing at the byte level. In any case, the relationship between the length in bits A of an address and the number N of addressable units is $2^A = N$.

- iii. **Unit of transfer:** For main memory, this is the number of bits read out of or written into memory at a time. **The unit of transfer need not equal a word or an addressable unit.** For external memory, data are often transferred in much larger units than a word, and these are referred to as blocks.

4. **Method of Accessing:** It is a fundamental characteristic of memory devices. It is the sequence or order in which memory can be accessed. These include the following:

- i. **Sequential access:** Memory is organized into units of data, called records. Access must be made in a specific linear sequence. Stored addressing information is used to separate records and assist in the retrieval process. A shared read–write mechanism is used, and this must be moved from its current location to the desired location, passing and rejecting each intermediate record.

Thus, the time to access an arbitrary record is highly variable. Tape units are an example sequential access.

- ii. **Direct access:** As with sequential access, direct access involves a shared read–write mechanism. However, individual blocks or records have a unique address based on physical location. Access is accomplished by direct access to reach general vicinity plus sequential searching, counting, or waiting to reach the final location. Again, access time is variable.

- iii. **Random access:** Each addressable location in memory has a unique, physically wired-in addressing mechanism. The time to access, a given location is independent of the sequence of prior accesses and is constant. Thus, any location can be selected

at random and directly addressed and accessed. Main memory and some cache systems are random access.

- iv. **Associative: This** is a random access type of memory that enables one to make a comparison of desired bit locations within a word for a specified match, and to do this for all words simultaneously. Thus, a word is retrieved based on a portion of its contents rather than its address. As with ordinary random-access memory, each location has its own addressing mechanism, and retrieval time is constant independent of location or prior access patterns. Cache memories may employ associative access.

5. **Performance.** One of the chief memory characteristic from the user's point of view is its performance, the other being the capacity. Three performance parameters are used:

- i. **Access time (latency):** For random-access memory, this is the time it takes to perform a read or write operation, that is, the time from the instant that an address is presented to the memory to the instant that data have been stored or made available for use. For non-random-access memory, access time is the time it takes to position the read-write mechanism at the desired location.
- ii. **Memory cycle time:** This concept is primarily applied to random-access memory and consists of the access time plus any additional time required before a second access can commence. This additional time may be required for transients to die out on signal lines or to regenerate data if they are read destructively.
Note that memory cycle time is concerned with the system bus, not the processor.
- iii. **Transfer rate:** This is the rate at which data can be transferred into or out of a memory unit. For random-Access memory, it is equal to $1/(\text{cycle time})$. For non-random-access memory, the following relationship holds:

$$T_n = T_A + \frac{n}{R}$$

Where

T_n = Average time to read or write n bits

T_A = Average access time

n = Number of bits

R = Transfer rate, in bits per second (bps)

- 6. **Physical types:** The most common today are semiconductor memory, magnetic surface memory, used for disk and tape, and optical and magneto-optical.
- 7. **Physical characteristics:** An important feature of data storage devices. In a **volatile memory**, information decays naturally or is lost when electrical power is switched off. In a **nonvolatile memory**, information once recorded remains without deterioration until deliberately changed; no electrical power is needed to retain information. Magnetic-surface memories are nonvolatile.

Semiconductor memory (memory on integrated circuits) may be either volatile or nonvolatile. Nonerasable memory cannot be altered, except by destroying the storage unit. Semiconductor memory of this type is known as *read-only memory* (ROM). Of necessity, a practical nonerasable memory must also be nonvolatile.

8. **Organization:** One of the key design issues in random access memories. In this context, *Organization* refers to the physical arrangement of bits to form words.

3.2 The Memory Hierarchy

In the Computer System Design, Memory Hierarchy is an enhancement to organize the memory such that it can minimize the access time. The Memory Hierarchy was developed based on a program behavior known as locality of references. The principles of locality of reference can be explained thus; during the course of execution of a program, memory references by the processor, for both instructions and data, tend to cluster. Programs typically contain a number of iterative loops and subroutines. Once a loop or subroutine is entered, there are repeated references to a small set of instructions. Similarly, operations on tables and arrays involve access to a clustered set of data words. Over a long period of time, the clusters in use change, but over a short period, the processor is primarily working with fixed clusters of memory references.

There is a trade-off among the three key characteristics of memory: capacity, access time, and cost. A variety of technologies is used to implement memory systems, and across this spectrum of technologies, the following relationships hold:

- Faster access time, greater cost per bit;
- Greater capacity, smaller cost per bit;
- Greater capacity, slower access time.

The dilemma facing the designer is clear. The designer would like to use memory technologies that provide for large- capacity memory, both because the capacity is needed and because the cost per bit is low. However, to meet performance requirements, the designer needs to use expensive, relatively lower-capacity memories with short access times.

The way out of this dilemma is not to rely on a single memory component or technology, but to employ a memory hierarchy.

Figure 3.1 illustrates a typical hierarchy. As one goes down the hierarchy, the following occur:

- a. Decreasing cost per bit;
- b. Increasing capacity;
- c. Increasing access time;
- d. Decreasing frequency of access of the memory by the processor.

Thus, smaller, more expensive, faster memories are supplemented by larger, cheaper, slower memories. The key to the success of this organization is item (d): decreasing frequency of access which is based on the principle of locality of reference stated above.

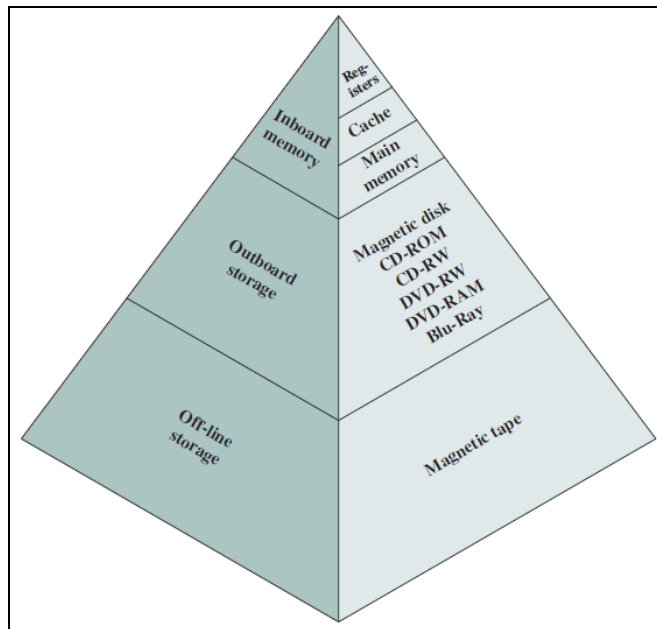


Figure 3.1: The Memory Hierarchy

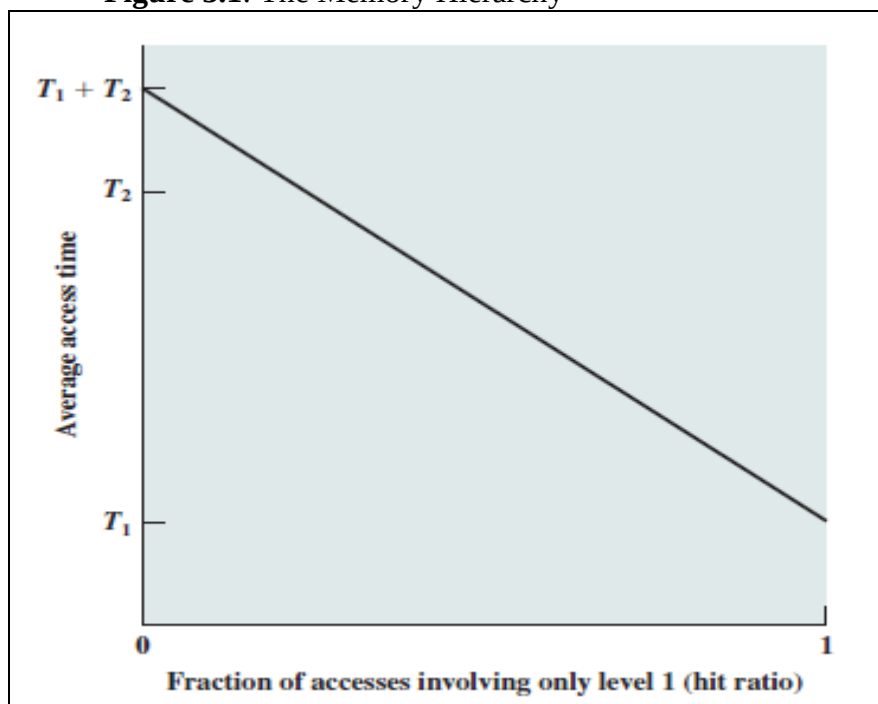


Figure 3.2: Performances of Access Involving only Level 1 (hit ratio)

This principle can be applied across more than two levels of memory, as suggested by the hierarchy shown in Figure 3.1. The fastest, smallest, and most expensive type of memory consists of the *registers internal to the processor*. Typically, a processor will contain a few dozen such registers, although some machines contain hundreds of registers. Main memory is the principal internal memory system of the computer. Each location in main memory has a unique address. Main memory is usually extended with a higher-speed, smaller cache. The cache is not usually visible to the programmer or, indeed, to the processor. It is a device for staging the movement of data between main memory and processor registers to improve performance. The three forms of memory just described are, typically, volatile and employ semiconductor technology. The use of three levels exploits the fact that semiconductor memory comes in a variety of types, which differ in speed and cost. Data are stored more

permanently on external mass storage devices, of which the most common are hard disk and removable media, such as removable magnetic disk, tape, and optical storage. External, nonvolatile memory is also referred to as secondary memory or auxiliary memory. These are used to store program and data files and are usually visible to the programmer only in terms of files and records, as opposed to individual bytes or words. Disk is also used to provide an extension to main memory known as virtual memory.

3.3 Cache Memory

Cache memory is an extremely fast semiconductor memory type that acts as a buffer between RAM and the CPU. It is used to speed up and synchronizing with high-speed CPU. Cache memory is costlier than main memory or disk memory but economical than CPU registers. It holds frequently requested data and instructions so that they are immediately available to the CPU when needed. The cache is a smaller and faster memory that stores copies of the data from frequently used main memory locations. Cache memory is used to reduce the average time to access data from the Main memory.

Cache memory is designed to combine the memory access time of expensive, high-speed memory combined with the large memory size of less expensive, lower-speed memory. The concept is illustrated in Figure 3.3a. There is a relatively large and slow main memory together with a smaller, faster cache memory. The cache contains a copy of portions of main memory. When the processor attempts to read a word of memory, a check is made to determine if the word is in the cache. If so, the word is delivered to the processor. If not, a block of main memory, consisting of some fixed number of words, is read into the cache and then the word is delivered to the processor. Because of the phenomenon of locality of reference, when a block of data is fetched into the cache to satisfy a single memory reference, it is likely that there will be future references to that same memory location or to other words in the block.

Figure 3.3b depicts the use of multiple levels of cache. The L2 cache is slower and typically larger than the L1 cache, and the L3 cache is slower and typically larger than the L2 cache. Figure 3.4 depicts the structure of a cache/main-memory system. Main memory consists of up to 2^n addressable words, with each word having a unique n -bit address.

For mapping purposes, this memory is considered to consist of a number of fixed-length blocks of K words each. That is, there are $M = 2^n/K$ blocks in main memory. The cache consists of m blocks, called lines. Each line contains K words, plus a tag of a few bits. Each line also includes control bits (not shown), such as a bit to indicate whether the line has been modified since being loaded into the cache. The length of a line, not including tag and control bits, is the **line size**. The line size may be as small as 32 bits, with each “word” being a single byte; in this case, the line size is 4 bytes. The number of lines is considerably less than the number of main memory blocks ($m \ll M$)

At any time, some subset of the blocks of memory resides in lines in the cache. If a word in a block of memory is read, that block is transferred to one of the lines of the cache. Because there are more blocks than lines, an individual line cannot be uniquely and permanently dedicated to a particular block. Thus, each line includes a **tag** that identifies which particular block is currently being stored.

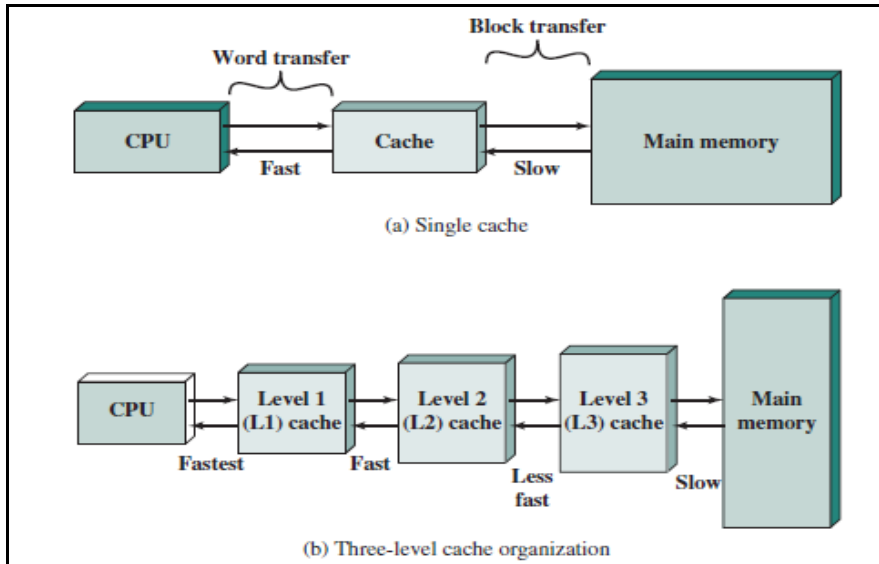


Figure 3.3: Cache and main Memory

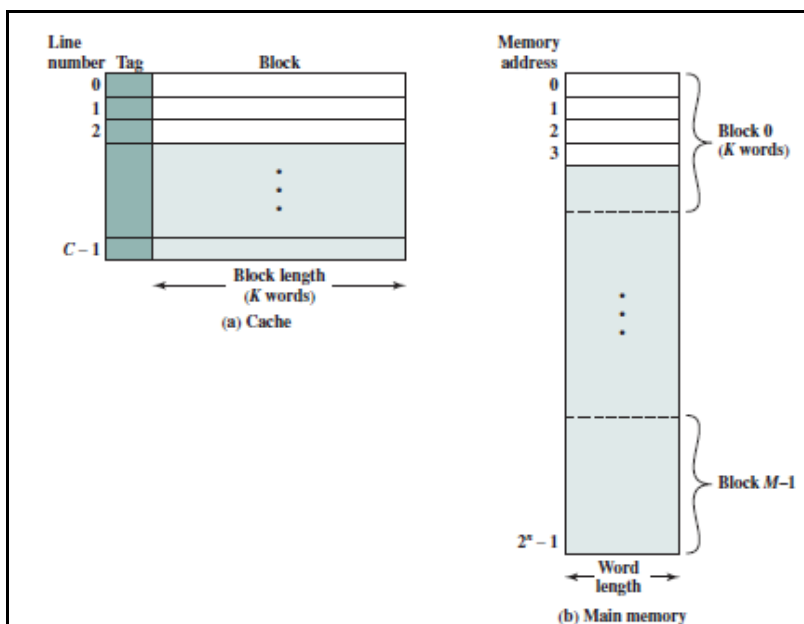


Figure 3.4: Cache/Main memory Structure

3.3.1 Cache Performance:

When the processor needs to read or write a location in main memory, it first checks for a corresponding entry in the cache.

- If the processor finds that the memory location is in the cache, a cache **hit** has occurred and data is read from cache
- If the processor does not find the memory location in the cache, a cache **miss** has occurred. For a cache miss, the cache allocates a new entry and copies in data from main memory, and then the request is fulfilled from the contents of the cache.

The performance of cache memory is frequently measured in terms of a quantity called Hit ratio.

Hit ratio = $\text{hit} / (\text{hit} + \text{miss}) = \text{no. of hits} / \text{total accesses}$

We can improve Cache performance using higher cache block size, higher associativity, reduce miss rate, reduce miss penalty, and reduce the time to hit in the cache.

3.3.2 Cache Read operation

Figure 3.5 illustrates the read operation. The processor generates the read address (RA) of a word to be read. If the word is contained in the cache, it is delivered to the processor. Otherwise, the block containing that word is loaded into the cache, and the word is delivered to the processor. Figure 3.5 shows these last two operations occurring in parallel and reflects the organization shown in Figure 3.6, which is typical of contemporary cache organizations.

In this organization, the cache connects to the processor via data, control, and address lines. The data and address lines also attach to data and address buffers, which attach to a system bus from which main memory is reached. When a cache hit occurs, the data and address buffers are disabled and communication is only between processor and cache, with no system bus traffic. When a cache miss occurs, the desired address is loaded onto the system bus and the data are returned through the data buffer to both the cache and the processor. In other organizations, the cache is physically interposed between the processor and the main memory for all data, address, and control lines. In this latter case, for a cache miss, the desired word is first read into the cache and then transferred from cache to processor.

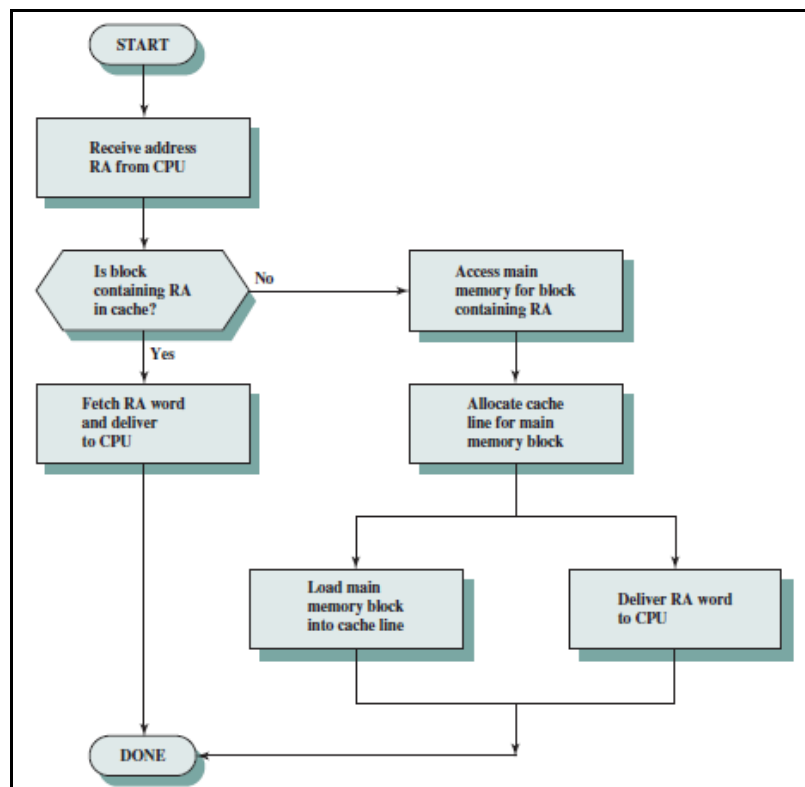


Figure 3.5: Cache Read Operation

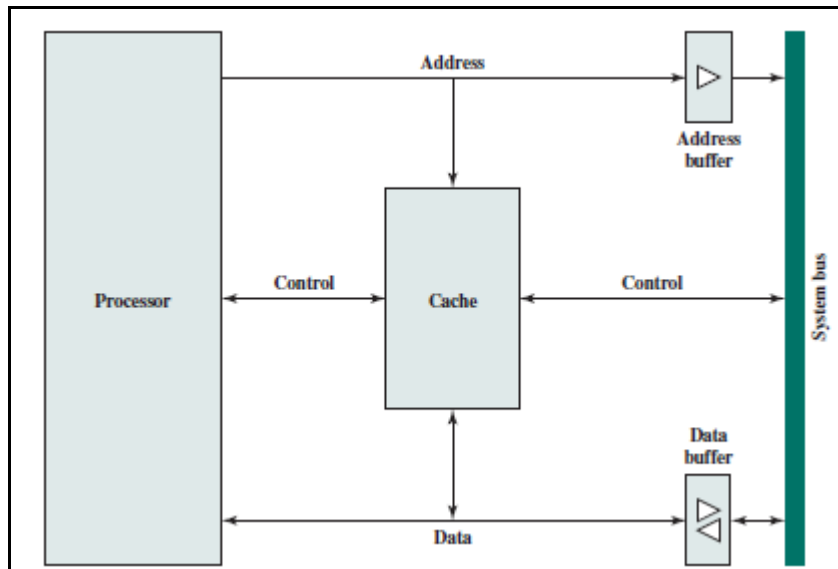


Figure 3.6: Typical Cache organization

3.3.3 Elements of Cache Design

This section provides an overview of cache design parameters. We occasionally refer to the use of caches in **high-performance computing (HPC)**.

Although there are a large number of cache implementations, a few basic design elements serve to classify and differentiate cache architectures. Figure 3.7 lists the key elements.

(1) Cache Addresses

Almost all non-embedded processors, and many embedded processors, support virtual memory. In essence, virtual memory is a facility that allows programs to address memory from a logical point of view, without regard to the amount of main memory physically available. When virtual memory is used, the address fields of machine instructions contain virtual addresses.

(2) Cache Size

We would like the size of the cache to be small enough so that the overall average cost per bit is close to that of main memory alone and large enough so that the overall average access time is close to that of the cache alone. There are several other motivations for minimizing cache size. The larger the cache, the larger the number of gates involved in addressing the cache.

(3) Mapping Function

Because there are fewer cache lines than main memory blocks, an algorithm is needed for mapping main memory blocks into cache lines. Further, a means is needed for determining which main memory block currently occupies a cache line. The choice of the mapping function dictates how the cache is organized. Three techniques can be used: **direct, associative, and set-associative.**

(4) Replacement Algorithms

Once the cache has been filled, when a new block is brought into the cache, one of the existing blocks must be replaced. For direct mapping, there is only one possible line for any particular block, and no choice is possible. For the associative and set-associative techniques, a replacement algorithm is needed. To achieve high speed, such an algorithm must be implemented in hardware. A number of algorithms have been tried. Four of the most common algorithms are stated here.

Probably the most effective is **Least Recently Used (LRU)**, others are: **FIFO (First-In First Out)**, **LFU (Least Frequently Used)**, and **RR (Random replacement)**.

(5) Write Policy

When a block that is resident in the cache is to be replaced, there are two cases to consider. If the old block in the cache has not been altered, then it may be overwritten with a new block without first writing out the old block. If at least one write operation has been performed on a word in that line of the cache, then main memory must be updated by writing the line of cache out to the block of memory before bringing in the new block. A variety of write policies, with performance and economic trade-offs, is possible.

(6) Line size:

Another design element is the line size. When a block of data is retrieved and placed in the cache, not only the desired word but also some number of adjacent words are retrieved. As the block size increases from very small to larger sizes, the hit ratio will at first increase because of the principle of locality, which states that data in the vicinity of a referenced word are likely to be referenced in the near future. As the block size increases, more useful data are brought into the cache. The hit ratio will begin to decrease, however, as the block becomes even bigger and the probability of using the newly fetched information becomes less than the probability of reusing the information that has to be replaced.

(7) Number of Caches

When caches were originally introduced, the typical system had a single cache. More recently, the use of multiple caches has become the norm. Two aspects of this design issue concern the number of levels of caches and the use of *unified versus split caches*.

Multilevel caches; as logic density increased, it has become possible to have a cache on the same chip as the processor: the on-chip cache. Compared with a cache reachable via an external bus, the on-chip cache reduces the processor's external bus activity and therefore speeds up execution times and increases overall performance.

Cache Addresses	Write Policy
Logical	Write through
Physical	Write back
Cache Size	Line Size
Mapping Function	Number of Caches
Direct	Single or two level
Associative	Unified or split
Set associative	
Replacement Algorithm	
Least recently used (LRU)	
First in first out (FIFO)	
Least frequently used (LFU)	
Random	

Figure 3.7 : Elements of Cache Designs